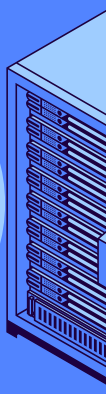
PATRÓN DE DISEÑO
FLYWEGHT

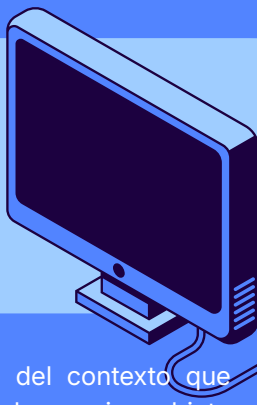


Tipo de patrón: Estructural

Significa que se enfoca en cómo se combinan, ensamblan y prganizan tanto clases como objetos para formar estructuras más grandes y complejas, optimizando su realción y arquitectura

Propósito

Utiliza la compartición para soportar grandes cantidades de objetos de grano fino (fine-grained objects) de manera eficienteLos objetos de grano fino (fine-grained objects) son objetos pequeños que representan entidades muy elementales, específicas y de bajo nivel dentro de un sistema.



¿Qué es un flyweight?

es un objeto compartido que puede usarse en múltiples contextos simultáneamente. Para lograrlo, divide el estado del objeto en dos partes:

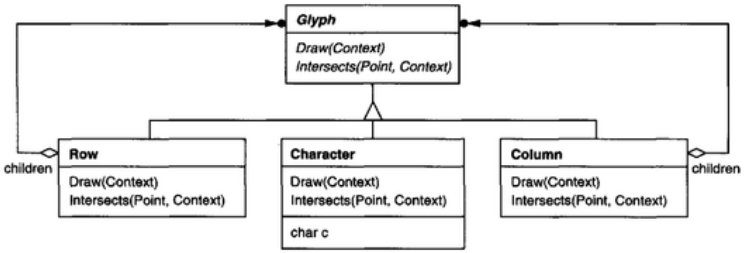
1. **Estado intrínseco:**
   Información independiente del contexto que se almacena dentro del propio objeto flyweight, haciéndolo compartible.

2. **Estado extrínseco:**
   Depende del contexto del objeto y varía según este, por lo que no se puede compartir y debe ser suministrado o calculado por los objetos cliente cuando se necesite.



DIAGRAMAS DE CLASE

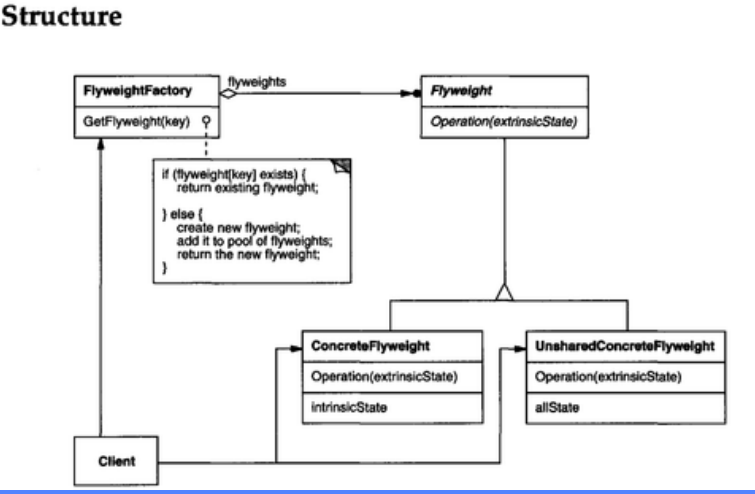
Clase Abstracta Principal (Glyph): Es la superclase que representa a los objetos gráficos, permitiendo que algunos de ellos actúen como flyweights.



De ***Design Patterns Elements of Reusable Object-Oriented Software:***

“se muestra la estructura de clases para estos objetos. Glyph es la clase abstracta para objetos gráficos, algunos de los cuales pueden ser flyweights. Las operaciones que pueden depender del estado extrínseco lo reciben como parámetro. Por ejemplo, Draw e Intersects deben saber en qué contexto está el glyph antes de poder hacer su trabajo."

Diagrama genérico de estructura del patrón Flyweight



Partes que lo componen:

FlyweightFactory (Fábrica):

Crea y gestiona los objetos flyweight a la vez que asegura que los objetos se compartan correctamente. Cuando un cliente solicita uno mediante GetFlyweight(key), la fábrica verifica si ya existe en su colección interna: si existe, lo devuelve; si no, lo crea, lo añade al conjunto y lo retorna.

1

Flyweight (Interfaz):

Declara la interfaz a través de la cual los objetos flyweight pueden recibir y actuar sobre el estado extrínseco. Su operación principal recibe el estado extrínseco como parámetro (Operation(extrinsicState)).

2

ConcreteFlyweight (Flyweight Concreto):

Implementa la interfaz Flyweight y añade almacenamiento para el estado intrínseco (intrinsicState). Los objetos de esta clase deben ser totalmente compartibles e independientes de su contexto.

3



UnsharedConcreteFlyweight (Flyweight No Compartido):

Subclases que no necesitan ni deben ser compartidas. El patrón permite su existencia para casos donde se requiere almacenar todo el estado internamente (allState).

4

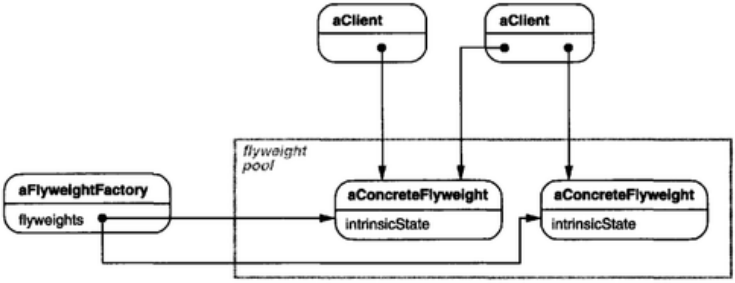
Client (Cliente):

- Mantiene referencias a los objetos flyweight y calcula o almacena su estado extrínseco.
- Regla clave: Los clientes no deben instanciar objetos ConcreteFlyweight directamente, sino obtenerlos exclusivamente a través de la FlyweightFactory.

5

Diagrama de objetos

Muestra cómo interactúan instancias reales en un momento dado:

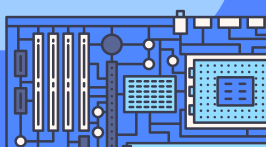


Partes que lo componen:

El Pool de Flyweights (Flyweight Pool):

Es el contenedor gestionado por la fábrica (aFlyweightFactory) que almacena de forma centralizada las instancias únicas de los objetos compartidos (aConcreteFlyweight), asegurando que no se dupliquen en memoria.

1



Compartición Simultánea:

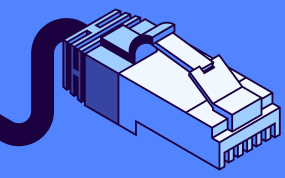
Se puede ver cómo dos clientes distintos (aClient) pueden apuntar y reutilizar el mismo objeto compartido (aConcreteFlyweight) que vive dentro del pool simultáneamente, sin estorbarse.

2

Separación de Estados:

Cada objeto en el pool solo guarda su estado intrínseco (intrinsicState), mientras que el contexto o estado que varía se maneja externamente por los clientes.

3



RELACIÓN CON OTROS PATRONES



Composite:

El patrón Flyweight se combina frecuentemente con el patrón Composite para representar una estructura jerárquica como un grafo con nodos hoja compartidos. Debido a esta compartición, los nodos hoja no pueden almacenar un puntero a su padre; dicho puntero se pasa como parte del estado extrínseco.

1

Factory Method (Método de Fábrica):

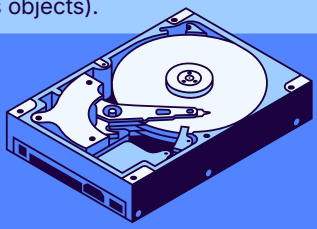
2

Se relaciona estrechamente porque la creación y gestión de los objetos compartidos suele delegarse en una fábrica (FlyweightFactory). Ya que esta parte en nuestro patrón es la encargada de asegurar que las instancias sean únicas y se reutilicen en lugar de crearse desde cero cada vez.

State (Estado) o Strategy (Estrategia):

En ocasiones, los objetos Flyweight pueden representar estados o estrategias que son compartidos por múltiples contextos (como diferentes objetos en un juego o componentes en una interfaz), ayudando a evitar la duplicación de objetos sin estado (stateless objects).

3



VENTAJAS Y DEVENTAJAS AL USAR FLYWEIGHT



VENTAJAS/PUNTOS POSITIVOS

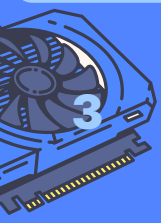
1

Ahorro masivo de espacio/memoria:
Se reduce drásticamente el número de instancias físicas en la aplicación al compartir los objetos.

Eficiencia en grano fino:

Permite modelar entidades muy pequeñas (como caracteres individuales en un texto) sin que el sistema colapse por falta de memoria.

2



3

El ahorro es mayor cuanto más estado se logra volver intrínseco (compartido) o cuando el estado extrínseco se calcula en lugar de almacenarse.

DESVENTAJAS/PUNTOS NEGATIVOS

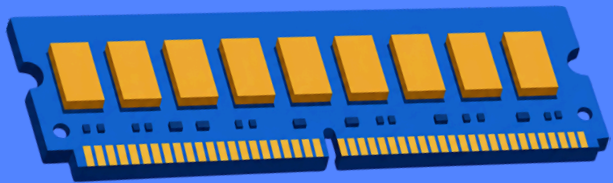
Coste en tiempo de ejecución (run-time overhead):
Puede introducir sobrecarga de procesamiento debido al tiempo necesario para buscar, transferir o calcular el estado extrínseco

1

Intercambio (trade-off):

2

Si decides calcular el estado extrínseco en lugar de guardarlo para ahorrar memoria, terminas intercambiando espacio de almacenamiento por tiempo de CPU.



CASOS DONDE

SE PUEDE APLICAR



Generalmente para usar el patrón estructural Flyweight se busca que se cumplan las siguientes condiciones:

1. **La aplicación utiliza un número masivo o exagerado de objetos.**
2. **Los costos de almacenamiento son muy elevados debido a la gran cantidad de objetos creados.**
3. **La mayor parte del estado del objeto puede convertirse en estado extrínseco (separándose del objeto compartido)**
   Para permitir la compartición
4. **Muchos grupos de objetos pueden ser reemplazados por una cantidad relativamente pequeña de objetos compartidos una vez que se extrae el estado dependiente del contexto.**
5. **La aplicación no depende de la identidad del objeto (dado que los objetos se comparten, las pruebas de identidad estricta devolverán true para instancias que conceptualmente son distintas).**

BIBLIOGRAFÍA USADA:

***Gamma, E., Helm, R., Johnson, R., & Vlissides, J.* (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.**

https://www.javier8a.com/itc/bd1/articulo.pdf